

9/8일 과제 1 C++ 보고서

과제 1에서 제일 중요한 조건은 모든 클래스가 **DataFrame**이라는 공통 클래스를 상속받아서 만들어져야 한다는 것이다.. **DataFrame**에는 데이터를 담는 포인터랑 크기 변수만 **protected**로 넣어두고, **size**랑 **clear**는 순수가상함수로 만들어서 상속받는 자식이 무조건 직접 구현하게 만들어져 있다.

나는 **queue**를 구현하기로 했다. C에서는 **queue**를 구현할 때 **linked list**로 했지만 **DataFrame**에 **T T* Data_**가 있는것으로 보아 동적 배열로 **queue**를 만들어야될 것 같았다. 처음엔 그냥 원소를 넣을 때마다 필요한 만큼만 새로 배열을 할당하려고 했는데, 이렇게 하면 **push**를 할 때마다 매번 새로 배열을 만들고 복사해야 해서 비효율적이라는 걸 알게 됐다. 그래서 배열이 꽉 찼을 때만 용량을 2배로 늘리는 방식으로 구현했다.

push_back처럼 뒤에 값을 추가하는 **push** 함수를 구현할 때, **front_**라는 변수를 따로 두어서 큐의 맨 앞이 배열의 몇 번째 인덱스인지 기억하도록 했다. **pop**을 할 때 배열 전체를 앞으로 밀지 않고 **front_**만 1 증가시키는 방식으로 구현했는데, 이렇게 하면 **pop** 하나에 걸리는 시간이 항상 일정해서 더 효율적이라고 생각했다. 대신 **pop**을 계속하면 배열 앞쪽에 이제 안 쓰는 공간이 계속 쌓이는 문제가 있었는데, 이건 **queue**가 꽉차서 배열을 재할당할 때 **front_**를 다시 0으로 리셋해주는 방식으로 해결했다.

그리고 **DataFrame**이 템플릿 클래스라서, **queue** 안에서 상속받은 **data_**나 **size_** 변수를 그냥 이름으로 쓰면 컴파일러가 못 찾는다는 것도 새로 알게 됐다. 그래서 **this->data_** 이런 식으로 꼭 **this**를 붙여줘야 했다. C++ 문법이 익숙하지 않고 처음 배우는 내용이다 보니 **this**를 안써서 계속 컴파일 에러가 나서 원인을 찾는데 시간이 좀 걸렸다.

원래 템플릿은 헤더 하나에 선언이랑 구현을 다 넣는게 정석이라고 하는데, 과제에서 파일을 나누라고 했기 때문에 **queue.cpp** 맨 아래에 **queue<int>**, **queue<double>**, **queue<char>**처럼 실제로 쓸 타입들을 미리 명시해주는 방식으로 해결했다. 이렇게 하면 여기 적어둔 타입 말고 다른 타입으로는 **queue**를 못 쓴다는 제약이 생기긴 하지만, 어쨌든 파일은 분리할 수 있었다.

```
template class queue<int>;  
template class queue<double>;  
template class queue<char>;
```

실행 결과는 다음과 같다. **push**로 1, 2, 3을 넣고 **front**, **back**, **size**를 확인한 뒤에 **pop**을 하고 다시 확인해봤다.

```
empty: 1
q size: 3
q fornt: 1
q back: 3
p size: 3
p fornt: 5
p back: 7
pop 후 front 2
pop 후 size: 2
swap 후
q size: 3
q fornt: 5
q back: 7
p size: 3
p fornt: 2
p back: 4
clear 후 empty: 1
```

front랑 back이 pop 전후로 잘 바뀌는 걸 보고 front_ 인덱스 방식이 제대로 동작한다는 걸 확인할 수 있고, swap을 통해 두 queue의 원소가 바뀌었다는 것을 확인 할 수 있다.